random subset of training data and computed the sum or average gradient for the batch. This averages over the targets and stabilizes the learning.

3.3.2. Experience replay

Catastrophic forgetting is probably not something we have to worry about with the first variant of our game because the targets are always stationary, and indeed the model successfully learned how to play it. But with the random mode, it's something we need to consider, and that is why we need to implement something called *experience replay*. Experience replay basically gives us batch updating in an online learning scheme. It's not a big deal to implement

Here's how experience replay works (figure 3.13):

1. In state s, take action a, and observe the new state s_{t+1} and reward r_{t+1}.
2. Store this as a tuple (s, a, s_{t+1}, r_{t+1}) in a list.
3. Continue to store each experience in this list until you have filled the list to a specific length (this is up to you to define).
4. Once the experience replay memory is filled, randomly select a subset (again, you need to define the subset size).
5. Iterate through this subset and calculate value updates for each subset; store these in a target array (such as Y) and store the state, s, of each memory in X.
6. Use X and Y as a mini-batch for batch training. For subsequent epochs where the array is full, just overwrite old values in your experience replay memory array.

Figure 3.13. This is the general overview of experience replay, a method for mitigating a major problem with online training algorithms: catastrophic forgetting. The idea is to employ mini-batching by storing past experiences and then using a random subset of these experiences to update the Q-network, rather than using just the single most recent experience.